A Speedup Theorem for Asynchronous Computation with Applications to Consensus and Approximate Agreement

Pierre Fraigniaud
pierre.fraigniaud@irif.fr
IRIF, CNRS & Université Paris Cité
Paris, France

Ami Paz
ami.paz@lisn.fr
LISN, CNRS & Université Paris Saclay
Paris, France

Sergio Rajsbaum
rajsbaum@im.unam.mx
Instituto de Matemáticas, UNAM
Mexico City, Mexico

ABSTRACT

We study two fundamental problems of distributed computing, *consensus* and *approximate agreement*, through a novel approach for proving lower bounds and impossibility results, that we call the *asynchronous speedup theorem*. For a given n-process task Π and a given computational model M, we define a new task, called the *closure* of Π with respect to M. The asynchronous speedup theorem states that if a task Π is solvable in $t \geq 1$ rounds in M, then its closure w.r.t. M is solvable in $t-1$ rounds in M. We prove this theorem for iterated models, as long as the model allows solo executions. We illustrate the power of our asynchronous speedup theorem by providing a new proof of the wait-free impossibility of consensus using read/write registers, and a new proof of the wait-free impossibility of solving consensus using registers and test&set objects for $n > 2$. The proof is merely by showing that, in each case, the closure of consensus (w.r.t. the corresponding model) is consensus itself. Our main application is the study of the power of additional objects, namely test&set and binary consensus, for wait-free solving approximate agreement *faster*. By analyzing the closure of approximate agreement w.r.t. each of the two models, we show that while these objects are more powerful than read/write registers from the computability perspective, they are not more powerful as far as helping solving approximate agreement faster is concerned.

CCS CONCEPTS

• **Theory of computation** → *Distributed computing models.*

KEYWORDS

Lower Bounds; Distributed Algorithms; Consensus; Approximate Agreement; Combinatorial Topology

ACM Reference Format:
Pierre Fraigniaud, Ami Paz, and Sergio Rajsbaum. 2022. A Speedup Theorem for Asynchronous Computation with Applications to Consensus and Approximate Agreement. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC '22), July 25–29, 2022, Salerno, Italy.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3519270.3538422

Pierre Fraigniaud received support from the ANR project DUCAT..
Part of this work was done while Sergio Rajsbaum was visiting LIX at École Polytechnique, and IRIF at Université Paris Cité, France. He received additional support from UNAM-PAPIIT IN106520.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '22, July 25–29, 2022, Salerno, Italy
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9262-4/22/07…$15.00
https://doi.org/10.1145/3519270.3538422

1 INTRODUCTION

One of the goals of the theory of distributed computing is to understand what can be achieved by a given distributed system. A great deal of research has been devoted to develop techniques for showing that certain problems cannot be solved, or to prove lower bounds on the resources needed to solve some given problem [3, 17, 32]. Much of this research is motivated by the need to understand the solvability of *consensus* and *approximate agreement*, two of the central tasks of distributed computing. In particular, this line of research lead to the introduction of bivalency [18] and indistinguishability [4] arguments, and later on to the discovery of the close connection between distributed computing and algebraic topology, based on the study of topological invariants of the protocol complex representing a given model of computation [24]. In this paper we propose a new approach to prove unsolvability results and time lower bounds.

1.1 Asynchronous Speedup Theorem

We introduce the *asynchronous speedup theorem* for shared-memory computing, inspired by the recent speedup technique [6, 12] for the LOCAL model of synchronous failure-free distributed computing in networks. Specifically, we consider the iterated shared-memory model, where the memory is organized in arrays M_r, $r \geq 1$, of n single-writer/multiple-readers (SWMR) registers (one per process), and each round r of the protocol is executed on the array M_r. Recall that a task $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ is defined by a collection $\mathcal{I}$ of possible input configurations, a collection $\mathcal{O}$ of possible output configurations, and a map $\Delta : \mathcal{I} \to 2^{\mathcal{O}}$ specifying an input-output relation. A subset of processes starting with inputs as specified by a configuration $\sigma \in \mathcal{I}$ must output values that define a configuration $\tau \in \Delta(\sigma)$.

For our speedup theorem, we define the *closure* of a task Π with respect to a computational model M, denoted $\mathsf{CL}_M(\Pi) = (\mathcal{I}, \mathcal{O}', \Delta')$, which is supposed to be a slightly easier version of Π under M. Recall that an algorithm solves a task Π in t rounds if, whenever a subset of processes start with inputs as specified by a configuration $\sigma \in \mathcal{I}$, after executing t rounds, they must output values that define a configuration $\tau \in \Delta(\sigma)$. Our asynchronous speedup theorem states that, for every $t \geq 1$, if a task Π is solvable in t rounds in M, then $\mathsf{CL}_M(\Pi)$ is solvable in $t-1$ rounds in M. Therefore, to establish a lower bound on the number t of rounds for solving Π, it suffice to show that iterating the closure operation for t times starting with Π, results in a task that is not solvable in zero rounds. Alternately, impossibility results follow when the closure operator can be performed infinitely many times without ever reaching a

task solvable in zero rounds, which is typically the case when the closure of a task is the task itself.

Notice that the notion of "$\mathrm{CL}_M(\Pi)$ being easier to solve than Π" is with respect to the model M of computation. We consider models obtained by extending the standard shared-memory model with objects more powerful than SWMR registers—the task $\mathrm{CL}_M(\Pi)$ then depends on the nature of these objects. Intuitively, $\mathrm{CL}_M(\Pi)$ is supposed to be easier than Π because it allows, on each given input configuration $\sigma \in \mathcal{I}$ to output some values that are illegal according to Δ. To prevent $\mathrm{CL}_M(\Pi)$ from being *too* easy in M, it is however required that such an output configuration τ is "close to a legal solution", in the following sense: there must exists a 1-round algorithm which, starting with the configuration τ as input, computes legal outputs for σ, that is, outputs a configuration in $\Delta(\sigma)$. If there is such a 1-round algorithm for M, then the configuration τ is added to the collection $\Delta'(\sigma)$ of legal output configurations for σ in the closure task $\mathrm{CL}_M(\sigma)$.

1.2 Our Results

We present our asynchronous speedup theorem for any iterated asynchronous model with SWMR registers, as long as this model allows *solo* executions (Theorem 1). Such models include the usual wait-free iterated immediate snapshot (IIS), iterated snapshot, and iterated collect models. They also include *affine models* [31] that allow solo executions, such as the k-concurrency model [21]. Roughly, an affine model is obtained from the IIS model by removing some executions. Iterated models with solo executions that *add* some executions have also been studied, such as the d-solo models [26], where d processes may run solo in the same execution. Note that lower bounds for iterated models, as the ones we prove here, immediately apply to the non-iterated variants of the same models—in a non-iterated model, the adversary can still choose to have only executions where all processes finish their r-th step before taking their $(r+1)$-th step.

We illustrate the use of our asynchronous speedup theorem by revisiting the renowned wait-free impossibility of solving consensus. We show how to derive this impossibility result in a simple way from our speedup theorem (Corollary 1): to prove the impossibility of consensus, it is enough to establish that the closure of consensus is consensus itself. Next, we present an extension of our speedup theorem for models as above augmented with objects more powerful than SWMR registers (Theorem 2). We illustrate its usefulness with a new proof of the impossibility for consensus among $n \geq 3$ processes when the processes have access to a test&set object at each round (Corollary 2).

After considering solvability, we move our attention to the question of how much objects in Herlihy's consensus hierarchy can help solving a problem *faster*. We present two examples of objects that are more powerful than read/write registers, namely test&set and binary consensus, and yet not more powerful as far as helping solving approximate agreement faster is concerned. Our proofs, based on our asynchronous speedup theorem, provide novel information about the topology of such extended models. More specifically, in the second part of the paper, we illustrate the power of the asynchronous speedup theorem by establishing new lower bounds for ϵ-approximate agreement, in the wait-free IIS model with access to test&set or binary consensus objects. We first show that the number of rounds required for solving ϵ-approximate agreement in the wait-free IIS model among $n \geq 3$ processes is at least $\lceil \log_2(1/\epsilon) \rceil$ (Corollary 3), which is tight, and then we show that augmenting the model with test&set objects does not reduce the time complexity at all (Theorem 3), assuming that each of the processes calls all the test&set objects. These results are obtained by simply showing that the closure of ϵ-approximate agreement in the considered model is the (2ϵ)-approximate agreement task. For $n = 2$, ϵ-approximate agreement can be solved in a single round in the wait-free IIS model with test&set, while in the wait-free IIS model without additional objects we show that ϵ-approximate agreement among $n = 2$ processes requires $\lceil \log_3 1/\epsilon \rceil$ rounds (Corollary 3). This bound is proved by showing that the closure of ϵ-approximate agreement in this case is the (3ϵ)-approximate agreement task, and is also known to be tight.

Finally, we show that the number of rounds required by $n \geq 3$ processes for solving the ϵ-approximate agreement task in the wait-free IIS model augmented with a binary consensus object is at least $\min\{\lceil \log_2 1/\epsilon \rceil, \lceil \log_2 n \rceil - 1\}$, when we assume that each input to the binary consensus object depends solely on the process ID and of the round number. In this context, the closure of ϵ-approximate agreement is not necessarily (2ϵ)-approximate agreement. However, we can show that, for a set of at least half of the processes, the closure of ϵ-approximate agreement when only the processes in this set participate is (2ϵ)-approximate agreement. By iterating this argument, we get that if there is a t-round algorithm for solving ϵ-approximate agreement among n processes, then there is a $t - r$ round algorithm for solving $(2^r\epsilon)$-approximate agreement among $n/2^r$ processes. We lose an additive factor -1 for technical reasons related to the fact that ϵ-approximate agreement is solvable when there are only $n = 2$ processes. Note that our lower bound is essentially tight as there exists an algorithm for solving ϵ-approximate agreement in $\lceil \log_2 1/\epsilon \rceil$ rounds in the wait-free IIS model [1], and there exists an algorithm for solving multi-value consensus in $\lceil \log_2 n \rceil$ rounds [34, 36] in the wait-free IIS model with binary consensus objects.

1.3 Related Work

The computability and complexity of fault-prone, asynchronous consensus and approximate agreement is a well studied topic. Fischer, Lynch, and Paterson (FLP) [18] showed that consensus cannot be solved in a message passing system even if only one process may fail by halting. Later, Herlihy [23] studied wait-free computation in the shared-memory model, and proved the consensus impossibility in this model. Motivated by the need to analyze problems other than consensus, Herlihy and Shavit [27] introduced the use of combinatorial topology for reasoning about computations in asynchronous, wait-free distributed systems, in which any number of processes may crash. They proved the asynchronous computability theorem, which states a topological condition that is necessary and sufficient for a task to be wait-free solvable using read/write registers. Approaches similar to the one of Herlihy and Shavit were presented in the same time by Saks and Zaharoglou [37] and by Borowsky and Gafni [9], but the formalization of Herlihy and Shavit is the one commonly used today. To conclude, the two techniques that have

been extensively used for proving impossibility results for consensus and approximate agreement are based on either valency [18] or connectivity analysis [27], in addition to indirect, reduction [23] or simulation techniques e.g. [14]. Our technique of computing the closure of consensus provides a novel way for proving such impossibility results. Regarding complexity issues, Hoest and Shavit [28] showed how the topological approach can also be used for deriving time lower bounds for wait-free algorithms in the IIS model, as we do. Attiya, Castañeda, Herlihy and Paz [2] used similar techniques for proving time upper bounds, and Ellen, Gelashvili, and Zhu [16] used them for proving space lower bounds. One aim of our paper is to investigate further the use of combinatorial topology in the context of asynchronous computing.

Soon after the celebrated FLP consensus impossibility result, Dolev, Lynch, Pinter, Stark, and Weihl [15] introduced the approximate agreement task, in which each process has a real valued input, and processes must agree on output values within the range of inputs, that are at most ϵ apart. Aspnes and Herlihy [1] proved a lower bound of $\lceil \log_3 1/\epsilon \rceil$ rounds and an upper bound of $\lceil \log_2 1/\epsilon \rceil$ rounds in the wait-free SWMR model, a gap that was later closed by Hoest and Shavit [28], who proved a tight bound on the number of rounds needed to solve approximate agreement: $\lceil \log_3 1/\epsilon \rceil$ for $n = 2$, and $\lceil \log_2 1/\epsilon \rceil$ for $n \geq 3$. A similar result was recently proved in the context of dynamic networks by Függer, Nowak and Schwarz [20], where the reader can find a thorough discussion about the literature studying approximate agreement, and about the importance of this problem w.r.t. applications. Hoest and Shavit [28] prove their lower bounds using a global analysis of the protocol complex, while Függer, Nowak and Schwarz [20] extend the notion of valency used for consensus [18, 33] for proving their results. We establish these lower bounds in a novel way, by computing the closure of the approximate agreement task.

In addition to proving the wait-free consensus impossibility, Herlihy [23] also derived a hierarchy of objects such that no object at one level has a wait-free implementation in terms of objects at lower levels, introducing a remarkable technique of reduction to a consensus protocol. There is plenty of work on extending the wait-free IIS model with more powerful objects, but only from the computability perspective, e.g. [22, 29] using simulations, or using topology e.g. [25]. We are not aware of any use of this technique for complexity results, to study as we do, using strong objects to solve approximate agreement faster.

Last, but not least, a goal of this research is to understand the potential extension of the recent speedup technique [6, 12] from the (synchronous failure-free) LOCAL model to asynchronous models with crash-prone processes. In the LOCAL model, the uncertainly about how a hypothetical protocol would solve a problem in one round, comes from unknown inputs of far away processes. In contrast, in the asynchronous setting, the uncertainty is of a very different nature, about what other processes have read. There have been few recent attempts to approach distributed network computing through the lens of algebraic topology [13, 19]. An extension of the speedup technique in [6, 12] to arbitrary round-based full-information models has been proposed in [7]. This extension applies to wait-free shared-memory computing in particular, but only for 2 processes. Our approach applies to an arbitrarily large number of processes.

2 MODEL

We consider the standard *read-write shared memory* distributed computing model [5], and we adopt a standard framework for modeling distributed computation, by using concepts borrowed from algebraic topology [24].

2.1 Iterated Models

We consider distributed systems with $n \geq 2$ asynchronous processes, labeled by distinct integers from 1 to n. Every process i initially knows its identity i as well as the total number n of processes in the system. We focus on generic round-based algorithms of the following form. All read and write operations are atomic. Specifically, Algorithm 1 is for the *iterated* model, in which the shared-memory is organized in arrays M_r, $r \geq 1$, of n single-writer/multiple-readers registers (one per process), and each round r of the protocol is executed on the array M_r. At each round, each process i updates its so-called *view* V_i by collecting the views of the other processes. To this end, it uses a *collect* operation, which consists of reading all registers $M_r[1..n]$ sequentially, in arbitrary order. The initial view of a process is its input, and, after t rounds or write-collect, each process outputs a value which depends of its view after t rounds, i.e., of all the information accumulated by the process during the execution of the algorithm.

Algorithm 1: *Code for process $i \in [n]$ with input x_i*

```
1 V_i ← x_i
2 for r = 1 to t
3 |   write (i, V_i) in register M_r[i]
4 |   V_i ← collect M_r[1..n]
5 end
6 y_i ← f(i, V_i)
7 output y_i.
```

Two classical stronger variants of the collect instruction are considered in this paper. *Snapshot* corresponds to the scenario in which the collect operation itself is atomic, that is, all registers $M_r[1..n]$ are globally read simultaneously at once, in an atomic manner [5]. *Immediate snapshot* [9, 37] corresponds to the scenario in which each write-snapshot sequence of operations is itself atomic, that is, not only all registers are read simultaneously at once, but the snapshot occurs "immediately" after the write operation: a set of processes first all perform a write operation, and immediately after they all perform an atomic snapshot (the order of reads inside the set of processes is not important, and so is the order of snapshot operations). Our lower bound technique applies to the three wait-free models with write-collect, write-snapshot, or immediate snapshot, and we shall establish our lower bound for approximate agreement using the stronger model, i.e., iterated immediate snapshot, or IIS for short.

2.2 Task Solvability

A *task* for n processes is a triple $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ where $\mathcal{I}$ and $\mathcal{O}$ are $(n-1)$-dimensional complexes, respectively called *input* and *output* complexes, and $\Delta : \mathcal{I} \to 2^{\mathcal{O}}$ is an input-output specification.

Every simplex $\sigma = \{(i, x_i) : i \in I\}$ of $\mathcal{I}$, where $I = \text{ID}(\sigma)$ is a non-empty subset of $[n]$, defines a legal input state corresponding to the scenario in which, for every $i \in I$, process i starts with input value x_i. Similarly, every simplex $\tau = \{(i, y_i) : i \in I\}$ of $\mathcal{O}$ defines a legal output state corresponding to the scenario in which, for every $i \in I$, process i outputs the value y_i. The map Δ is an input-output relation specifying, for every input state $\sigma \in \mathcal{I}$, the set of output states $\tau \in \mathcal{O}$ with $\text{ID}(\tau) = \text{ID}(\sigma)$ that are legal with respect to σ. That is, assuming that only the processes in $\text{ID}(\sigma)$ participate to the computation (the set of participating processes is not known a priori to the processes in σ), these processes are allowed to output any simplex $\tau \in \Delta(\sigma)$. $\Delta(\sigma)$ can be viewed as a collection of simplexes, each with the same set of IDs as σ, or, alternatively, as the complex induced by this set of simplexes. It is often assumed that Δ is a *carrier* map (that is, for every $\sigma, \sigma' \in \mathcal{I}$, if $\sigma' \subseteq \sigma$ then $\Delta(\sigma') \subseteq \Delta(\sigma)$ as subcomplexes), but in this paper we do not enforce this requirement into the definition of tasks.

Given a simplex $\sigma = \{(i, x_i) : i \in I\} \in \mathcal{I}$ for some $I = \text{ID}(\sigma) \subseteq [n]$, one round of communication performed by the processes in $\text{ID}(\sigma)$ as in Algorithm 1 results in various possible simplices, depending on the interleaving of the different write and read operations. Such a simplex is of the form $\tau = \{(i, V_i) : i \in I\}$, where $V_i = \{(j, x_j) : j \in J_i\}$ is the *view* of process i after one round. This view, or equivalently, the set $J_i \subseteq I$, depends on the communication model M, i.e., write-collect, write-snapshot, or immediate snapshot. These simplices induces a complex, denoted by $\mathcal{P}^{(1)}(\sigma)$, whose structure differs according to the three models considered on this paper. In the case of immediate snapshot, $\mathcal{P}^{(1)}(\sigma)$ is the complex obtained by performing a chromatic subdivision of σ [27]. That is, $\tau = \{(i, V_i) : i \in I\}$ is in $\mathcal{P}^{(1)}(\sigma)$ if and only if, for every $i, j \in I$, $j \in V_i$ or $i \in V_j$, and if $j \in V_i$ then $V_j \subseteq V_i$. We denote by Ξ the map transforming every $\sigma \in \mathcal{I}$ into $\mathcal{P}^{(1)}(\sigma)$, and by $\mathcal{P}^{(1)} = \cup_{\sigma \in \mathcal{I}} \mathcal{P}^{(1)}(\sigma)$. The map Ξ depends on the communication model M.

The topological transformation from σ to $\mathcal{P}^{(1)}(\sigma)$ can be iterated, yielding the sequence $(\mathcal{P}^{(t)})_{t \geq 0}$ where, for every simplex $\sigma \in \mathcal{I}$, $\mathcal{P}^{(0)}(\sigma) = \sigma$, and, for every $t \geq 1$, $\mathcal{P}^{(t)}(\sigma) = \Xi(\mathcal{P}^{(t-1)}(\sigma))$. For every input complex $\mathcal{I}$, and every $t \geq 0$, the complex $\mathcal{P}^{(t)}$ is called the *protocol complex* after t rounds. This complex is the union, for all simplices $\sigma \in \mathcal{I}$ of the complex $\mathcal{P}^{(t)}(\sigma)$ resulting from t rounds starting from input σ. Note that for any two input simplices $\sigma = \{(i, x_i) : i \in I\}$ and $\sigma' = \{(i, x'_i) : i \in I\}$, the complexes $\mathcal{P}^{(1)}(\sigma)$ and $\mathcal{P}^{(1)}(\sigma')$ are isomorphic. We denote by

$$\chi : \mathcal{P}^{(1)}(\sigma) \to \mathcal{P}^{(1)}(\sigma') \tag{1}$$

the canonical isomorphism that maps every vertex $v = (i, \{(j, x_j) : j \in J_i\})$ of $\mathcal{P}^{(1)}(\sigma)$ to the vertex $\chi(v) = (i, \{(j, x'_j) : j \in J_i\})$ of $\mathcal{P}^{(1)}(\sigma')$. Finally, recall that a task $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ is solvable in t rounds if and only if there exists a simplicial map $f : \mathcal{P}^{(t)} \to \mathcal{O}$ from the t-round protocol complex to the output complex that agrees with Δ, i.e., for every $\sigma \in \mathcal{I}$, $f(\mathcal{P}^{(t)}(\sigma)) \subseteq \Delta(\sigma)$. The mapping f is defined by the vertices, i.e., for $\sigma = \{(i, V_i) : i \in I\}$ we have $f(\sigma) = \{f(i, V_i) : i \in I\}$. The simplicial map f is merely the function f used in Algorithm 1 for computing the output values of the processes.

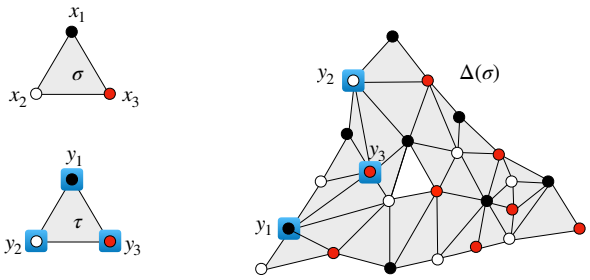


Figure 1: *Local task*

3 CLOSURE AND SPEEDUP THEOREM

This section describes our main tool for deriving lower bounds and impossibility results, namely an *asynchronous speedup theorem*. This theorem is based on a notion of *closure* defined hereafter.

3.1 A Closure of a Task

In this section, we define the notion of a *closure task* (Figure 2). As it will be shown later, given a computational model M, and given a task Π, the closure of Π w.r.t. M, denoted by $\text{CL}_M(\Pi)$, is a simpler task in the sense that if Π is solvable in t rounds in model M, then $\text{CL}_M(\Pi)$ is solvable in $t - 1$ rounds in model M. In order to define the closure of a task, we first define another task, called *local task* (Figure 1), which does not depend on M, but only on Π. Intuitively, this task closes the one-round gap between the round-complexities of $\text{CL}_M(\Pi)$ and Π by taking the outputs of $\text{CL}_M(\Pi)$ as input, and producing legal outputs for Π.

Let $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ be a task. We say that a set $\tau \subseteq V(\mathcal{O})$ of output vertices is *chromatic* if, for any two vertices v and v' of τ, $\text{ID}(v) \neq \text{ID}(v')$. Intuitively, to every simplex $\sigma \in \mathcal{I}$, and to every chromatic set of output vertices $\tau \subseteq V(\mathcal{O})$ satisfying $\text{ID}(\tau) = \text{ID}(\sigma)$, we associate a specific task with input τ, viewed as a complex, and whose objective is to construct a legal solution in $\Delta(\sigma)$. Note that τ is a chromatic set of vertices in $V(\mathcal{O})$, but is not necessarily a simplex in $\mathcal{O}$. Given a chromatic complex $\mathcal{K}$ with vertex IDs in $[n]$, and given a non-empty set $I \subseteq [n]$, we denote by $\text{proj}_I(\mathcal{K})$ the subcomplex of $\mathcal{K}$ induced by the vertices with IDs in I.

Definition 1. *Given a task $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$, a simplex $\sigma \in \mathcal{I}$, and a chromatic set $\tau \subseteq V(\Delta(\sigma))$ satisfying $\text{ID}(\tau) = \text{ID}(\sigma)$, the* local task *for σ and τ is the task $\Pi_{\tau,\sigma} = (\tau, \Delta(\sigma), \Delta_{\tau,\sigma})$ where $\Delta_{\tau,\sigma} : \tau \to 2^{\Delta(\sigma)}$ is the map defined by:*

1. *for every vertex $v \in \tau$: $\Delta_{\tau,\sigma}(v) = \{v\}$;*
2. *for every $\tau' \subseteq \tau$ with $|\tau'| > 1$: $\Delta_{\tau,\sigma}(\tau') = \text{proj}_{\text{ID}(\tau')}(\Delta(\sigma))$.*

Figure 1 provides an illustration of a local task, and in particular of the map $\Delta_{\tau,\sigma}$. This map is very similar to Δ applied to σ, but with one important feature: the vertices of τ are fixed by $\Delta_{\tau,\sigma}$, in the sense that a process i of τ running solo must output its value y_i in τ. On the other hand, $\Delta_{\tau,\sigma}$ is more liberal than Δ since, for a face τ' of τ with dimension greater than 0, the image of τ' by $\Delta_{\tau,\sigma}$ is any simplex of $\Delta(\sigma)$ colored with the IDs of the processes in τ'.

Remark. Given the local task $\Pi_{\tau,\sigma} = (\tau, \Delta(\sigma), \Delta_{\tau,\sigma})$ and $\tau' \subseteq \tau$, the set $\Delta_{\tau,\sigma}(\tau')$ induces a subcomplex of $\Delta(\sigma)$, potentially reduced

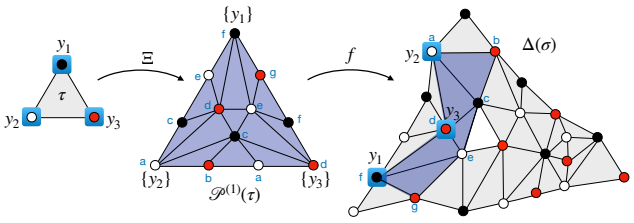


Figure 2: *Closure of a task*

to a single vertex whenever $|\tau'| = 1$. In particular, if τ is not a single vertex then $\Delta_{\tau,\sigma}(\tau) = \Delta(\sigma)$.

We are now ready to define the closure task $\text{CL}_M(\Pi)$, which will be a task that is expected to be easier than Π for model M. Given an input simplex σ for Π, the closure of Π allows to output chromatic sets of vertices $\tau \subseteq V(\Delta(\sigma))$ which are not simplices of $\Delta(\sigma)$. However, this set τ must be "close to each other" in $\Delta(\sigma)$, in the sense that the local task $\Pi_{\tau,\sigma}$ can be solved in one round w.r.t. a given model M.

Definition 2. *The* closure *of a task $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ with respect to a communication model M is the task $\text{CL}_M(\Pi) = (\mathcal{I}, \mathcal{O}', \Delta')$ where $V(\mathcal{O}') = V(\mathcal{O})$, and, for any $\sigma \in \mathcal{I}$ and any chromatic set $\tau \subseteq V(\mathcal{O})$ satisfying $\text{ID}(\tau) = \text{ID}(\sigma)$, $\tau \in \Delta'(\sigma)$ if and only if $\tau \subseteq V(\Delta(\sigma))$, and the local task $\Pi_{\tau,\sigma}$ is solvable in at most one round in M. The simplices of $\mathcal{O}'$ are the images of Δ', and all their faces.*

Figure 2 illustrates Definition 2, and in particular of how the carrier map Δ' is built. In this figure, the closure is with respect to the immediate snapshot model, as witnessed by the fact that the complex $\mathcal{P}^{(1)}(\tau)$ is the standard chromatic subdivision. We have $\tau \in \Delta'(\sigma)$ because the local task $\Pi_{\tau,\sigma} = (\tau, \Delta(\sigma), \Delta_{\tau,\sigma})$ is solvable in one round with immediate snapshot, as there is a simplicial map $f : \mathcal{P}^{(1)}(\tau) \to \Delta(\sigma)$ that respects $\Delta_{\tau,\sigma}$: the chromatic subdivision of τ is mapped to the dark subcomplex of $\Delta(\sigma)$ (the small letters specify f). Note that the closure of a task depends on the communication model, as a local task $\Pi_{\tau,\sigma}$ may be solvable in one round of some model M, but not of some other model M'.

Remark. Given a task $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ and its closure $\text{CL}_M(\Pi) = (\mathcal{I}, \mathcal{O}', \Delta')$ w.r.t. some model M, we have $\Delta(\sigma) \subseteq \Delta'(\sigma)$ for every $\sigma \in \mathcal{I}$. To see this, consider some $\sigma \in \mathcal{I}$ and let $\tau \subseteq V(\mathcal{O})$ be a chromatic set satisfying $\text{ID}(\tau) = \text{ID}(\sigma)$. If $\tau \in \Delta(\sigma)$ (that is, τ is a simplex of $\Delta(\sigma)$) then the local task $\Pi_{\tau,\sigma} = (\tau, \Delta(\sigma), \Delta_{\tau,\sigma})$ is solvable in zero rounds, by having each process output its input.

Since $\Delta(\sigma) \subseteq \Delta'(\sigma)$ for every $\sigma \in \mathcal{I}$, the closure $\text{CL}_M(\Pi) = (\mathcal{I}, \mathcal{O}', \Delta')$ of a task $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ is not more difficult that the task itself. In the next subsection, we will show that the closure of a task is in fact *simpler* than the original task.

3.2 Asynchronous Speedup Theorem

We say that an iterated model M *allows solo executions*, if in every round t of Algorithm 1, for each process i, there is an execution where the operations of process i in round t take place before the operations of all the other processes in this round. Thus, there is

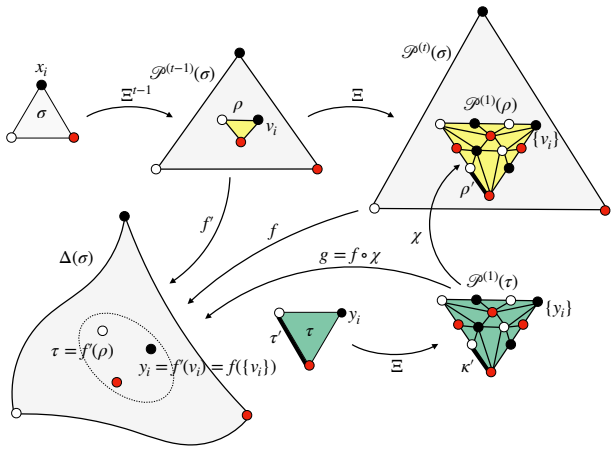


Figure 3: *Construction in the proof of Theorem 1*

an execution where process i reads only its own write in its collect operation.

Theorem 1. *Let M be an iterated model allowing solo executions. For every $t \geq 1$, if a task Π is solvable in t rounds in M, then the closure of Π with respect to M is solvable in $t-1$ rounds in M.*

Proof. Let $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ be a task on n processes, let $t \geq 1$, and assume that Π is solvable in t rounds in model M. Let $\text{CL}_M(\Pi) = (\mathcal{I}, \mathcal{O}', \Delta')$ be the closure of Π w.r.t. M. Our goal is to show that $\text{CL}_M(\Pi)$ is solvable in $t-1$ rounds in M.

Since Π is solvable in t rounds, there exists a simplicial map $f : \mathcal{P}^{(t)} \to \mathcal{O}$ that agrees with Δ. We aim at defining a simplicial map $f' : \mathcal{P}^{(t-1)} \to \mathcal{O}'$, that agrees with Δ'. To this end, let $(i, V_i) \in \mathcal{P}^{(t-1)}$ be a vertex of $\mathcal{P}^{(t-1)}$, i.e., V_i is a possible view of process $i \in [n]$ after $t-1$ rounds (see Figure 3). We merely define $f'(i, V_i) = f(i, \{(i, V_i)\})$. That is, $f'(i, V_i)$ is equal to the image by f of the vertex $(i, \{(i, V_i)\}) \in \mathcal{P}^{(t)}$ resulting from $(i, V_i) \in \mathcal{P}^{(t-1)}$ whenever process i runs solo at round t. Note that $f(i, \{(i, V_i)\})$ is a vertex of $\mathcal{O}$, and thus $f'(i, V_i)$ is a vertex of $\mathcal{O}'$, as $V(\mathcal{O}') = V(\mathcal{O})$.

Let us show that f' is simplicial and agrees with Δ'. To this end, let $\sigma \in \mathcal{I}$, and let $\rho \in \mathcal{P}^{(t-1)}(\sigma)$, i.e., ρ results from input σ after $t-1$ rounds of communication (see Figure 3). We aim at establishing that $f'(\rho)$ is a simplex of $\Delta'(\sigma)$. Let us assume that $\sigma = \{(i, x_i) : i \in I\}$ and $\rho = \{(i, V_i) : i \in I\}$, for some set $I \subseteq [n]$ of indices. Let us then consider $\tau = f'(\rho)$. For every $i \in I$, let $(i, y_i) = f'(i, V_i)$ be the output vertex returned by f' for process i with view V_i in $\mathcal{P}^{(t-1)}(\sigma)$. We have $\tau = f'(\rho) = \{f'(i, V_i) : i \in I\} = \{(i, y_i) : i \in I\}$. To establish that f' is simplicial and agrees with Δ', it is sufficient to show that τ is a simplex of $\Delta'(\sigma)$.

To show that $\tau \in \Delta'(\sigma)$, it is sufficient to show that the local task $\Pi_{\tau,\sigma} = (\tau, \Delta(\sigma), \Delta_{\tau,\sigma})$ is solvable in one round (cf. Definition 2), which we show by defining a decision map $g : \mathcal{P}^{(1)}(\tau) \to \Delta(\sigma)$ as follows (see Figure 3). Formally, the vertices of $\mathcal{P}^{(1)}(\tau)$ are in one-to-one correspondence with the vertices of $\mathcal{P}^{(1)}(\varrho)$, due to the canonical isomorphism χ between $\mathcal{P}^{(1)}(\tau)$ and $\mathcal{P}^{(1)}(\varrho)$ (cf. Eq. (1)), and so we define $g = f \circ \chi$. Operationally, this map g corresponds to the following algorithm for solving the local task $\Pi_{\tau,\sigma}$ in one

round. For each $i \in \text{ID}(\tau)$, process i starts with input y_i. It performs one communication round and collects a set $z_i = \{(j, y_j) : j \in J_i\}$ with $i \in J_i \subseteq I$. To compute its output value, process i uses solely the set J_i of indices appearing in z_i (and not the values collected), simulates an execution of the tth round of communication starting from ρ in which it gathers the view V_j, for every $j \in J_i$, resulting in a view $W_i = \{(j, V_j) : j \in J_i\}$. Then, process i outputs $f(i, W_i)$.

The map g is simplicial since f and χ are simplicial. It remains to show that g agrees with $\Delta_{\tau,\sigma}$. Two conditions must be fulfilled (cf. Definition 1). First, let (i, y_i) be a vertex of τ for some $i \in \text{ID}(\tau)$. We have $g(i, \{(i, y_i)\}) = f(i, \{(i, V_i)\}) = f'(i, V_i) = (i, y_i)$. It follows that $g(i, \{(i, y_i)\}) \in \Delta_{\tau,\sigma}(i, y_i)$ since, by definition, $\Delta_{\tau,\sigma}(i, y_i) = \{(i, y_i)\}$, and Condition 1 of Definition 1 is fulfilled. To check Condition 2, let $\tau' \in \tau$ be a simplex of the input complex τ of the local task $\Pi_{\tau,\sigma}$. In order to show that $g(\mathcal{P}^{(1)}(\tau')) \subseteq \Delta_{\tau,\sigma}(\tau')$, consider a simplex $\kappa' \in \mathcal{P}^{(1)}(\tau')$ with $\text{ID}(\kappa') = \text{ID}(\tau')$, and let $\varrho' = \chi(\kappa')$ be the corresponding simplex in $\mathcal{P}^{(1)}(\varrho)$. By the definition of the map g, we have $g(\kappa') = f \circ \chi(\kappa') = f(\varrho')$. Since f solves the task Π with input $\sigma \in \mathcal{I}$, we have $f(\varrho') \in \Delta(\sigma)$. Since $\text{ID}(f(\rho')) = \text{ID}(\varrho')$, we get that $f(\varrho') \in \text{proj}_{\text{ID}(\tau')}(\Delta(\sigma))$ as desired, and Condition 2 of Definition 1 holds as well. It follows that the map g does solve the local task $\Pi_{\tau,\sigma}$ in a single round, and therefore $\tau \in \Delta'(\sigma)$. As a consequence, f' is simplicial and agrees with Δ', which completes the proof. □

3.3 Impossibility of Consensus in the wait-free IIS Model

The speedup theorem enables to provide short proofs for classical impossibility results or lower bounds, as illustrated below for Consensus. A task $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ is said to be a *fixed-point* for model M whenever $\text{CL}_M(\Pi) = \Pi$.

Lemma 1. *A task Π that is a fixed-point for model M is either solvable in zero rounds in M, or unsolvable in M.*

Proof. Let us assume that there exists $t \geq 1$ such that Π is solvable in t rounds in M. By the speedup theorem, $\text{CL}_M(\Pi) = \Pi$ is solvable in $t - 1$ rounds in M. By repeating the application of the speedup theorem to $t - 1, t - 2, \ldots, 1$, we eventually get that Π is solvable in zero rounds in M. □

Recall that binary consensus task for n processes is the task $(\mathcal{I}, \mathcal{O}, \Delta)$ defined a follows. The input complex $\mathcal{I}$ is composed of all simplices of the form $\sigma = \{(i, x_i) : i \in I\}$ for some nonempty set $I \subseteq [n]$ of processes, where $x_i \in \{0, 1\}$ for every $i \in I$. The output complex $\mathcal{O}$ has two facets τ_0 and τ_1, where, for $y \in \{0, 1\}$, $\tau_y = \{(i, y) : i \in [n]\}$. The set of legal output simplices for the input simplex $\sigma = \{(i, x_i) : i \in I\} \in \mathcal{I}$ is

$$\Delta(\sigma) = \begin{cases} \{\text{proj}_I(\tau_0), \text{proj}_I(\tau_1)\} & \text{if there exists } i, j \in I \text{ s.t. } x_i \neq x_j \\ \{\sigma\} & \text{otherwise.} \end{cases}$$

Corollary 1. *Binary consensus is impossible to solve in the wait-free IIS model.*

Proof. Let Π denote the binary consensus task, and let M denote the wait-free IIS model. For establishing the impossibility result, it is sufficient to show that $\text{CL}_M(\Pi) = \Pi$. After proving this, the impossibility of consensus is immediate from Lemma 1, and the fact that binary consensus is not solvable in zero rounds, simply because, in consensus, every process running solo must output its input value, and, in a zero-round algorithm, every process must output the same as if it was running solo. Hence, our goal is to show that $\Delta'(\sigma) = \Delta(\sigma)$ for every $\sigma = \{(i, x_i) : i \in I\} \in \mathcal{I}$. Since, as noticed before, $\Delta(\sigma) \subseteq \Delta'(\sigma)$ for every task, it suffices to show $\Delta'(\sigma) \subseteq \Delta(\sigma)$ for binary consensus.

Let $\tau = \{(i, y_i) : i \in I\} \subseteq V(\Delta(\sigma))$ be such that $\tau \in \Delta'(\sigma)$, and let us show that $\tau \in \Delta(\sigma)$. Assume first that $\sigma = \{(i, x) : i \in I\}$ for some $x \in \{0, 1\}$. In this case, $\Delta(\sigma) = \sigma$, and thus, since $\tau \subseteq V(\Delta(\sigma))$, we have $\tau = \sigma$, and thus $\Delta'(\sigma) = \Delta(\sigma)$. Assume now that σ contains two vertices with distinct input values 0 and 1. Our goal is to show that τ cannot include two vertices (i, y_i) and (j, y_j) with $y_i \neq y_j$. To this end, let $i, j \in I$ be two different indices. Since $\tau \in \Delta'(\sigma)$, the local task $\Pi_{\tau,\sigma} = (\tau, \Delta(\sigma), \Delta_{\tau,\sigma})$ is solvable in a single round. So, let $f : \mathcal{P}^{(1)}(\tau) \to \Delta(\sigma)$ be a simplicial map that agrees with $\Delta_{\tau,\sigma}$. Consider the path

$$(i, \{(i, y_i)\}),\ (j, \{(i, y_i), (j, y_j)\}),\ (i, \{(i, y_i), (j, y_j)\}),\ (j, \{(j, y_j)\})$$

connecting the vertices $(i, \{(i, y_i)\})$ and $(j, \{(j, y_j)\})$ in $\mathcal{P}^{(1)}(\tau)$. This path is composed of three distinct edges, i.e., three distinct 1-dimensional simplices, e_1, e_2, and e_3. Since f is simplicial, the image of this path by f is a path connecting the vertices $f(i, \{(i, y_i)\})$ and $f(j, \{(j, y_j)\})$. Since f agrees with $\Delta_{\tau,\sigma}$, it holds that $f(i, \{(i, y_i)\}) = (i, y_i)$ and $f(j, \{(j, y_j)\}) = (j, y_j)$. Moreover, each of the (not necessarily distinct) edges $f(e_k)$, $k \in \{1, 2, 3\}$, must belong to $\Delta_{\tau,\sigma}(\{(i, y_i), (j, y_j)\}) = \text{proj}_{\{i,j\}}(\Delta(\sigma))$. We have $\text{proj}_{\{i,j\}}(\Delta(\sigma)) = \{\text{proj}_{\{i,j\}}(\tau_0), \text{proj}_{\{i,j\}}(\tau_1)\}$. It follows that either all edges $f(e_1)$, $f(e_2)$, and $f(e_3)$ are equal to $\text{proj}_{\{i,j\}}(\tau_0)$, or all of them are equal to $\text{proj}_{\{i,j\}}(\tau_1)$. As a consequence, $y_i = y_j$. Therefore, all the output values in τ are identical, and thus $\tau \in \Delta(\sigma)$, as desired. □

4 EXTENSIONS OF THE SPEEDUP THEOREM

In order to design lower bounds or impossibility results to cases where processes have access to an object solving some specific tasks (e.g., test-and-set or binary consensus), we need to extend the speedup theorem to models that use not only registers but also other communication objects, which we informally call *black box* objects.

4.1 Augmented Models

We consider generic round-based algorithms in a model augmented with a black box object $\mathbf{B}$ as displayed in Algorithm 2. This algorithm is similar to Algorithm 1 except that a call to a black box $\mathbf{B}$ is inserted at every round between the write and collect instructions. The calls performed at two different rounds are independent, i.e., there are t copies $\mathbf{B}_1, \ldots, \mathbf{B}_t$ of $\mathbf{B}$, where $\mathbf{B}_r$ is the black box used by all processes at round r. Some of the black box objects we use have input and output values, hence before invoking $\mathbf{B}$ at a given round r, each process $i \in [n]$ computes an input value a_i for $\mathbf{B}_r$, which is chosen by a function α that takes into account the process i, its view V_i, and the round number r. When process i invokes $\mathbf{B}_r$ with input a_i, it gets a value b_i in return. To complete the round, process i collects the view of the other processes, and forms a new view which is the pair formed by the value obtained from $\mathbf{B}_r$, and the collection of the views of the other processes.

Algorithm 2: *Code for process $i \in [n]$ with input x_i*

```
V_i ← x_i
for r ← 1 to t
    write (i, V_i) to M_r[i]
    a_i ← α(i, V_i, r)
    b_i ← B_r(a_i)
    C_i ← collect M_r[1..n]
    V_i ← (b_i, C_i)
end
y_i ← f(i, V_i)
output y_i.
```

Remark. In this paper, we assume that the black box object **B** is *consistent* in the sense that, for the same inputs to the processes, and for the same interleaving of the reads and writes by the processes, **B** returns the same outputs to the processes. For instance, for the binary consensus box where some processes provide the box with input 0, and some others provide the box with input 1, the box will systematically produce the same outputs, either all 0s, or all 1s, for the same interleaving. We may assume consistency since we are only interested in lower bounds.

4.2 Extended Speedup Theorem

We say that an iterated model M augmented with a black box *allows solo executions*, if in every round t of Algorithm 2, for each process i, there is an execution where the operations of process i in round t take place before the operations of all other processes in this round.

Theorem 2. *Let M be an iterated model augmented with a black box allowing solo executions. For every $t \geq 1$, if a task Π is solvable in t rounds in M, then the closure of Π with respect to M is solvable in $t-1$ rounds in M.*

Proof. The proof is almost identical to the proof of Theorem 1, so we just underline the changes. Given a simplicial map $f : \mathcal{P}^{(t)} \to O$ that agrees with Δ, we define a simplicial map $f' : \mathcal{P}^{(t-1)} \to O'$, that agrees with Δ'. Let $(i, V_i) \in \mathcal{P}^{(t-1)}$ be a vertex of $\mathcal{P}^{(t-1)}$. We define $f'(i, V_i) = f\big(i, (b_i, \{(i, V_i)\})\big)$, where $b_i = \mathbf{B}_t(a_i)$ is the output of the black box **B** invoked at round t with $a_i = \alpha(i, V_i, t)$ as input, whenever process i is running solo in **B**. Let us show that f' is simplicial and agrees with Δ'. Let $\sigma = \{(i, x_i) : i \in I\} \in \mathcal{I}$ and $\varrho = \{(i, V_i) : i \in I\} \in \mathcal{P}^{(t-1)}(\sigma)$, and let $\tau = f'(\varrho) = \{(i, y_i) : i \in I\}$. To show that $\tau \in \Delta'(\sigma)$, let us consider the local task $\Pi_{\tau,\sigma} = (\tau, \Delta(\sigma), \Delta_{\tau,\sigma})$, and let us check that it is indeed solvable in one round. As in the proof of Theorem 1, we define $g : \mathcal{P}^{(1)}(\tau) \to \Delta(\sigma)$ as $g = f \circ \chi$ where $\chi : \mathcal{P}^{(1)}(\tau) \to \mathcal{P}^{(1)}(\varrho)$ is the canonical isomorphism. More specifically, every process $i \in I$ writes y_i in memory, calls the black box with input $a_i = \alpha(i, V_i, t)$ to get a value $b_i = \mathbf{B}_1(a_i) = \mathbf{B}_t(a_i)$ and collects a set $z_i = \{(j, y_j) : j \in J_i\}$ for some set J_i with $i \in J_i \subseteq I$. Process i outputs $g\big(i, (b_i, z_i)\big) = f\big(i, (b_i, W_i)\big)$ where $W_i = \{(j, V_j) : j \in J_i\}$. The fact that g is simplicial and agrees with $\Delta_{\tau,\sigma}$ follows exactly from the same arguments as in the proof of Theorem 1, using the fact that the box is consistent — which guarantees that, for every $\kappa \in \mathcal{P}^{(1)}(\tau)$, the outputs of the box for κ and for $\chi(\kappa) \in \mathcal{P}^{(1)}(\varrho)$ are identical. □

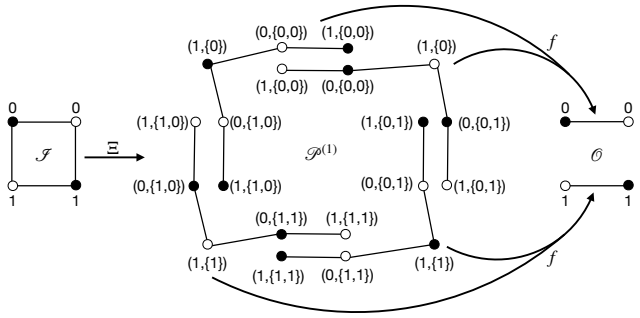


Figure 4: *2-process binary consensus is solvable using* test&set

4.3 Impossibility of Consensus in the Wait-Free IIS Model with test&set

Recall that test&set is an object that requires no inputs, and every process invoking test&set gets a value 0 or 1, with the guarantee that only the first process to invoke it gets a 1, all the other getting 0. It is known that test&set has consensus number 2 [23], i.e., it can be used to solve binary consensus among two processes, but not among more processes. Multi-value (i.e., not only binary) consensus among two processes can actually be solved in a single round with test&set: A process receiving the value 1 from test&set outputs its own input value, and a process receiving the value 0 from test&set outputs the input value of the other process. Note that a process i receiving the value 0 from test&set has the guarantee that the input value of the other process was written in memory before process i performs a snapshot, as if it was not running test&set solo (if the process would have run test&set solo, it would have obtained the value 1 from test&set).

Figure 4 illustrates why 2-process binary consensus is solvable with access to a test&set object. After one round, every process $i \in \{1, 2\}$ which sees only itself when reading the shared memory must win in test&set, and thus gets a view $(1, \{(i, x_i)\})$, where $x_i \in \{0, 1\}$ is the input of process i. Every process i which saw the other process j gets a view $(b_i, \{(i, x_i), (j, x_j)\})$ where x_i and x_j are the input values of processes i and j, respectively, and $b_i \in \{0, 1\}$ is the value produced by test&set for process i. The mapping $f : \mathcal{P}^{(1)} \to O$ displayed on Figure 4 is simplicial, and its composition with the map $\Xi : \mathcal{I} \to \mathcal{P}^{(1)}$ agrees with the specification of consensus.

In contrast, we show that for more than two processes, binary consensus is not solvable even using test&set, by applying Theorem 2. Figure 5 describes the protocol complex $\mathcal{P}^{(1)}(\sigma)$ for a 2-dimensional simplex σ for the immediate snapshot model augmented with test&set. For each $i \in \{1, 2, 3\}$, instead of four vertices with same ID i as in the 12-vertex chromatic subdivision of the triangle resulting from immediate snapshot, Figure 5 displays seven vertices with the same ID i, each vertex of the chromatic subdivision being duplicated depending on the outcome 0 or 1 of test&set, except the vertex corresponding to a solo execution for which the outcome of test&set is necessarily 1.

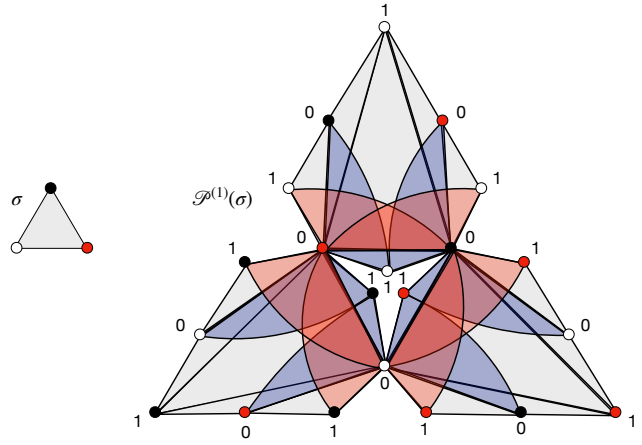


Figure 5: *1-round protocol complex for three processes using* test&set

Corollary 2. *For every $n > 2$, binary consensus among n processes is impossible to solve in the wait-free IIS model augmented with* test&set.

Proof. Let M be the model of iterated immediate snapshot with a test&set object. We cannot proceed exactly as in the proof of Corollary 1, as the binary consensus task $\Pi_{\text{CON}} = (\mathcal{I}, \mathcal{O}, \Delta)$ for $n > 2$ processes is not formally a fixed point for M. The reason for this is that, in Π_{CON}, even if only two processes i and j participate, they must agree on the same value, while in the closure task $\text{CL}_M(\Pi_{\text{CON}}) = (\mathcal{I}, \mathcal{O}', \Delta')$ this is not the case. Formally, since 2-process consensus is solvable using test&set in one round, for $\sigma = \{(i,0),(j,1)\}$ we have $\Delta'(\sigma) = \left\{\{(i,y_i),(j,y_j)\} : (y_i,y_j) \in \{0,1\}^2\right\}$.

Let us instead consider the simpler task $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$, where the input and output values and complexes are as in consensus, the validity condition must still be fulfilled (every output value must be an input value), but, only if at least three processes participate, they must output the same value. Every consensus algorithm also solves this problem, so it is enough to prove the impossibility of Π. We now prove that Π is a fixed point of M.

Let $\text{CL}_M(\Pi) = (\mathcal{I}, \mathcal{O}', \Delta')$ be the closure task of Π. Let us fix simplices $\sigma = \{(i, x_i) \mid i \in I\} \in \mathcal{I}$ and $\tau = \{(i, y_i), \mid i \in I\} \in \Delta'(\sigma)$ for a non-empty set $I \subseteq [n]$. If $|I| \leq 2$, then any subset of $\tau' \subseteq V(\Delta(\sigma))$ is already a simplex of $\Delta(\sigma)$, and we are done. Let us now consider the case of $|I| \geq 3$. Let $i, j, k \in I$, and let y_i, y_j, y_k be the values associated with the three processes i, j, k in τ. Our goal is to show that $y_i = y_j = y_k$, which will imply that $\tau \in \Delta(\sigma)$.

By the definition of a closure of a task, we know that the local task $\Pi_{\tau,\sigma} = (\tau, \Delta(\sigma), \Delta_{\tau,\sigma})$ is solvable in one round. Let $f : \mathcal{P}^{(1)} \to \mathcal{O}$ be a simplicial map solving $\Pi_{\tau,\sigma}$. For every $v = (\ell, y_\ell) \in \tau$, $\ell \in I$, we have $\Delta_{\tau,\sigma}(v) = \{v\}$, hence $f\big(\ell, (t_\ell, \{\ell, y_\ell)\})\big) = (\ell, y_\ell)$, where t_ℓ is the output of test&set, which is actually equal to 1 since the vertex $\big(\ell, (t_\ell, \{(\ell, y_\ell)\})\big)$ represents a solo step by process ℓ.

In $\mathcal{P}^{(1)}(\sigma)$, let us consider the two simplices (see Figure 6)

$$\varrho_{i,j,k} = \left\{ \begin{pmatrix} (i, (1, \{(i, y_i)\})), \\ (j, (0, \{(i, y_i), (j, y_j)\})), \\ (k, (0, \{(i, y_i), (j, y_j), (k, y_k)\})) \end{pmatrix} \right\},$$

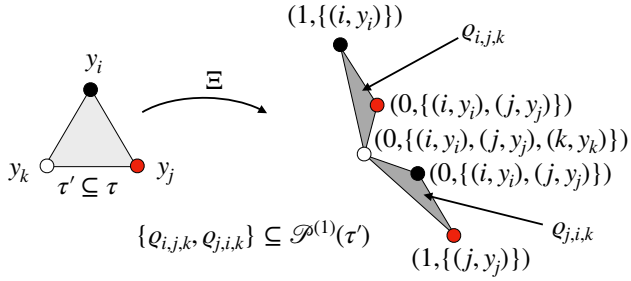


Figure 6: *Consensus is not solvable among $n > 2$ processes, even using* test&set

and

$$\varrho_{j,i,k} = \left\{ \begin{pmatrix} (j, (1, \{(j, y_j)\})), \\ (i, (0, \{(i, y_i), (j, y_j)\})), \\ (k, (0, \{(i, y_i), (j, y_j), (k, y_k)\})) \end{pmatrix} \right\}.$$

Let $\gamma = f\big(k, (0, \{(i, y_i), (j, y_j), (k, y_k)\})\big)$ be the output of f on the vertex $\big(k, (0, \{(i, y_i), (j, y_j), (k, y_k)\})\big)$ of $\mathcal{P}^{(1)}(\sigma)$. Since f agrees with $\Delta_{\tau,\sigma}$, we have that $f(\varrho_{i,j,k}) \subseteq \text{proj}_{\{i,j,k\}}(\Delta(\sigma))$, and the definition of Δ implies that, in $\Delta(\sigma)$, all processes output the same value $y_i = \gamma$. A similar argument on $f(\varrho_{j,i,k})$ implies that $y_j = \gamma$, i.e., $y_i = y_j$. By applying this argument for every $\ell \in I$, we get that all processes decide the same value γ in τ, that is, $\tau = \{(\ell, \gamma) \mid \ell \in I\}$, which implies that $\tau \in \Delta(\sigma)$, as claimed.

The rest of the proof is similar to the proof of Lemma 1, but using Theorem 2. That is, if Π was solvable in t rounds in M among $n \geq 3$ processes, then Π would have also been solvable in $t-1$ rounds, and, similarly, in 0 rounds. Since this is not the case, Π is not solvable in M at all. As mentioned above, Π is simpler than consensus, and so, for $n \geq 3$ processes, consensus is not solvable. □

5 LOWER BOUNDS FOR APPROXIMATE AGREEMENT

In this section, we apply our speedup theorem, and its extension, to the *approximate agreement* task. This task is a relaxation of the consensus problems, where processes have to decide on values that in the interval defined by the input values, and are close to one another (but not necessarily identical as in consensus). Given a (small) real value $\epsilon > 0$, in order to accurately define the ϵ-approximate agreement task while using only finite topological objects, we assume that ϵ and all the input values are rational numbers in the interval $[0, 1]$. Specifically, we discretize this interval as follows. We choose a (large) integer $m \geq 1/\epsilon$ such that ϵ is an integral multiple of $1/m$ (i.e., $m\epsilon$ is an integer), and so are all the possible input values of the task. We make sure that all the output values are also integral multiples of $1/m$. To this end, we avoid the usage of averaging algorithms, which are common in other works on approximate agreement. We formally define ϵ approximate agreement as follows.

Definition 3. *Given a parameter $\epsilon \in (0, 1]$ which is an integral multiple of $1/m$ for some integer $m \geq 1$, the* ϵ-approximate

agreement *for n processes is the task* $(\mathcal{I}, \mathcal{O}, \Delta)$*, where*

$$\mathcal{I} = \left\{ \{(i, x_i) : i \in I\} \;\middle|\; \begin{array}{l} (\emptyset \neq I \subseteq [n]) \wedge \\ (\forall i \in I,\ x_i \in \{0, 1/m, 2/m, \ldots, 1\}) \end{array} \right\}$$

$$\mathcal{O} = \left\{ \{(i, y_i) : i \in I\} \;\middle|\; \begin{array}{l} (\emptyset \neq I \subseteq [n]) \wedge \\ (\forall i \in I,\ y_i \in \{0, 1/m, 2/m, \ldots, 1\}) \wedge \\ (\forall i, j \in I,\ |y_i - y_j| \leq \epsilon) \end{array} \right\}$$

$\Delta : \mathcal{I} \to 2^{\mathcal{O}}$ *where* $\Delta\big(\{(i, x_i) : i \in I\}\big) =$

$$\left\{ \{(i, y_i) : i \in I\} \in \mathcal{O} \;\middle|\; \forall j \in I,\ \min\{x_i : i \in I\} \leq y_j \leq \max\{x_i : i \in I\} \right\}$$

We are interested in solving approximate agreement when the processes have access to an object that can be used in order to solve consensus among two processes, and hence also ϵ-approximate agreement for every ϵ. This creates a technical difficulty when applying our speed up technique, even when more than two processes are present. To overcome this, we define a relaxed version of ϵ-approximate agreement, where if exactly two processes participate, then they only need to output values in the range of inputs, but there is no bound on the distance between them.

Definition 4. *The* liberal version of ϵ-approximate agreement *is identical to the ϵ-approximate agreement task, except that the output complex $\mathcal{O}$ contains also all* 1*-dimensional chromatic simplices, regardless of the distances between their values. The map Δ specifying the task is formally defined the same, which means that for* 1*-dimensional input simplices it now contains more allowed output simplices.*

In fact, this is the type of relaxation we apply on the consensus task in the proof of Corollary 2. Note that any algorithm solving ϵ-approximate agreement also solves the liberal version of ϵ-approximate agreement, hence any lower bound for the latter implies a lower bound for the former.

5.1 Approximate Agreement in the Wait-Free IIS Model

Using our tools, we can easily reprove the fact that ϵ-approximate agreement takes $\Omega(\log 1/\epsilon)$ rounds in the standard IIS model [28]. For this, we show that the closure of ϵ-approximate agreement for two processes is (3ϵ)-approximate agreement, and (2ϵ)-approximate agreement for more processes. We start by proving some claims — the first of them is almost a triviality.

Claim 1. *For $\epsilon < 1$, ϵ-approximate agreement is not solvable in* 0 *rounds, and so is the liberal version of ϵ-approximate agreement for $n \geq 3$ processes.*

Proof. Let $\epsilon < 1$, and assume for contradiction that there is a 0-round ϵ-approximate agreement algorithm, or a 0-round algorithm solving the liberal version of ϵ-approximate agreement among $n \geq 3$ processes. Consider a setting with inputs $(1, 0)$ and $(2, 1)$, i.e., where process 1 has input 0 and process 2 has input 1. In a solo-execution each process must output its own input, and the setting where several processes run is indistinguishable to them from the one where each runs solo, so they will output 0 and 1 in all cases, contradicting the task definition. Formally, we have $\Delta(\{(1, 0)\}) = \{(1, 0)\}$ and $\Delta(\{(2, 1)\})) = \{(2, 1)\}$. As $\Xi : \mathcal{I} \to \mathcal{P}^{(0)}$ is simply the identity map, in a 0-round protocol any decision map f goes directly from $\mathcal{I}$ to $\mathcal{O}$. Hence, any decision map $f : \mathcal{I} \to \mathcal{O}$ must satisfy $f(1, 0) = (1, 0)$ and $f(2, 1) = (2, 1)$.

For the original ϵ-approximate agreement task, this implies that the input simplex $\sigma = \{(1, 0), (2, 1)\}$, is mapped to $f(\sigma) = \{(1, 0), (2, 1)\} \notin \Delta(\sigma)$ (in fact, we even have $f(\sigma) \notin \mathcal{O}$). For the liberal version of the task and $n \geq 3$, consider the same setting but with an extra "dummy" process 3, with input 0. As before, $f(3, 0) = (3, 0)$, and $f(\{(1, 0), (2, 1), (3, 0)\}) = \{(1, 0), (2, 1), (3, 0)\} \notin \Delta(\sigma)$, where here Δ is the specification of the liberal version of ϵ-approximate agreement. Hence, in both cases no algorithm and decision map f can solve the problem. □

The next claim is about the closure of approximate agreement in the specific case of two processes.

Claim 2. *If Π is the ϵ-approximate agreement task for two processes, then $\mathrm{CL}_M(\Pi)$ w.r.t. the wait-free IIS model M is the (3ϵ)-approximate agreement task for two processes.*

Proof. Let $\Pi_\epsilon = (\mathcal{I}, \mathcal{O}_\epsilon, \Delta_\epsilon)$ be the ϵ-approximate agreement task for $n = 2$ processes, and let $\mathrm{CL}_M(\Pi_\epsilon) = (\mathcal{I}, \mathcal{O}'_\epsilon, \Delta'_\epsilon)$ be its closure in the considered model M. Let us denote the (3ϵ)-approximate agreement task by $\Pi_{3\epsilon} = (\mathcal{I}, \mathcal{O}_{3\epsilon}, \Delta_{3\epsilon})$. We want to show that, for every $\sigma \in \mathcal{I}$, $\Delta'_\epsilon(\sigma) = \Delta_{3\epsilon}(\sigma)$. We consider separately the cases where σ is a vertex, and σ is an edge (recall that we are in the setting involving $n = 2$ processes).

Let $\sigma \in \mathcal{I}$ be a vertex. Let $\tau \in \Delta'_\epsilon(\sigma)$, which immediately implies $\tau \in \Delta_\epsilon(\sigma)$. As σ is a vertex, we have $\Delta_\epsilon(\sigma) = \{\sigma\}$, which implies $\tau = \sigma$. We also have $\Delta_{3\epsilon}(\sigma) = \{\sigma\}$, and hence $\tau \in \Delta_{3\epsilon}(\sigma)$. Conversely, let $\tau \in \Delta_{3\epsilon}(\sigma)$, and again $\Delta_{3\epsilon}(\sigma) = \{\sigma\}$ implies $\tau = \sigma$. The local task $(\sigma, \Delta_\epsilon(\sigma), \Delta_{\sigma,\sigma})$ is trivially solvable in one round—it is even solvable in zero rounds using the mapping $f(\sigma) = \sigma$—and hence $\tau \in \Delta'_\epsilon(\sigma)$. We conclude that for vertices, Δ'_ϵ and $\Delta_{3\epsilon}$ are identical.

Let $\sigma = \{(1, x_1), (2, x_2)\} \in \mathcal{I}$ be an edge. We consider an edge $\tau = \{(1, y_1), (2, y_2)\} \in \Delta'_\epsilon(\sigma)$, and show $\tau \in \Delta_{3\epsilon}(\sigma)$. To this end, we show (1) $\min\{x_1, x_2\} \leq y_i \leq \max\{x_1, x_2\}$ for all $i \in \{1, 2\}$, and (2) $|y_1 - y_2| \leq 3\epsilon$. Since $\tau \in \Delta'_\epsilon(\sigma)$, the local task $\Pi_{\tau,\sigma} = (\tau, \Delta_\epsilon(\sigma), \Delta_{\tau,\sigma})$ is solvable in one round, and let $f : \mathcal{P}^{(1)}(\tau) \to \Delta_\epsilon(\sigma)$ be a simplicial map solving it. Since $\tau \subseteq V(\Delta_\epsilon(\sigma))$, we know that every vertex $v = (i, y_i) \in \tau$, $i \in \{1, 2\}$, satisfies $\min\{x_1, x_2\} \leq y_i \leq \max\{x_1, x_2\}$, and thus property (1) holds. Let us denote by γ_1, γ_2 the two values satisfying

$$\begin{array}{rcl} f(1, \{(1, y_1)\}) & = & (1, y_1), \\ f(2, \{(1, y_1), (2, y_2)\}) & = & (2, \gamma_2), \\ f(1, \{(1, y_1), (2, y_2)\}) & = & (1, \gamma_1), \\ f(2, \{(2, y_2)\}) & = & (2, y_2). \end{array}$$

Since f solves the local task $\Pi_{\tau,\sigma} = (\tau, \Delta_\epsilon(\sigma), \Delta_{\tau,\sigma})$, we have $|y_2 - y_1| \leq |y_2 - \gamma_1| + |\gamma_1 - \gamma_2| + |\gamma_2 - y_1| \leq 3\epsilon$. Property (2) thus holds as well, implying $\tau \in \Delta_{3\epsilon}(\sigma)$.

Conversely, let us consider $\tau = \{(1, y_1), (2, y_2)\} \in \Delta_{3\epsilon}(\sigma)$, and let us assume, w.l.o.g., that $y_1 \leq y_2$. Let $f : \mathcal{P}^{(1)}(\tau) \to \Delta_\epsilon(\sigma)$ defined as follows, by setting $z = \min\{y_2, y_1 + \epsilon\}$, and

$$\begin{array}{rcl} f(1, \{(1, y_1)\}) & = & (1, y_1), \\ f(2, \{(1, y_1), (2, y_2)\}) & = & (2, z), \\ f(1, \{(1, y_1), (2, y_2)\}) & = & (1, \min\{y_2, z + \epsilon\}), \\ f(2, \{(2, y_2)\}) & = & (2, y_2). \end{array} \tag{2}$$

The map f solves ϵ-approximate agreement whenever $y_2 - y_1 \leq 3\epsilon$, and therefore it solves the local task $\Pi_{\tau,\sigma} = (\tau, \Delta_\epsilon(\sigma), \Delta_{\tau,\sigma})$ in one round. It follows that $\tau \in \Delta'_\epsilon(\sigma)$. We conclude that, for edges as well, Δ'_ϵ and $\Delta_{3\epsilon}$ are identical. □

The next claim is about the closure of the liberal version of approximate agreement in the case of three or more processes. Its proof resembles the proof of Claim 2, and appears in the full version of this paper.

Claim 3. *If $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ is the liberal version of ϵ-approximate agreement with $n \geq 3$ processes, then the closure $\mathrm{CL}_M(\Pi)$ of Π with respect to the wait-free IIS model M is the liberal version of (2ϵ)-approximate agreement.*

We are now ready to prove a lower bound for approximate agreement.

Corollary 3. *The number of rounds required for solving the ϵ-approximate agreement task in the wait-free IIS model with n processes is at least $\lceil \log_3(1/\epsilon) \rceil$ if $n = 2$, and $\lceil \log_2(1/\epsilon) \rceil$ if $n > 2$.*

Proof. Fix ϵ and n, and let Π denote the ϵ-approximate agreement task. Consider an algorithm solving ϵ-approximate agreement in t rounds. If $n = 2$, then, by Theorem 1 and Claim 2, there is an algorithm solving (3ϵ)-approximate agreement in $t - 1$ rounds. By repeating this argument t times, we get a $(3^t\epsilon)$-approximate agreement algorithm running in 0 rounds. By Claim 1, we have $3^t\epsilon \geq 1$, which immediately implies the lower bound. If $n \geq 3$, first note that the alleged algorithm also solves the liberal version of ϵ-approximate agreement in t rounds. By the same proof as above, but using Claim 3 instead of Claim 2, we get that there is a 0-round algorithm solving the liberal version of $(2^t\epsilon)$-approximate agreement, implying $2^t\epsilon \geq 1$, and the lower bound follows. □

Remark. The bounds in Corollary 3 are known to be tight [28]. In fact, the tightness of our lower bounds is also directly implied by our proofs, as specified, e.g. by Equation 2. This equation is for two processes, and essentially divides the distances by 3 at each round. A similar equation for the case of three or more processes appears in the proof of Claim 3, and essentially halves the distances at each round.

5.2 Approximate Agreement in the Wait-Free IIS Model Augmented with test&set

We now turn our attention to the approximate agreement task in an augmented version of the previous model, namely, we consider a model of iterated immediate snapshot augmented with test&set. It is known that the consensus number of test&set is 2, and, as explained in the previous section, in the model of immediate snapshot augmented with test&set, consensus among two processes can be solved in a single round. This immediately implies that the simpler task of approximate agreement among two processes is also solvable in a single round in this model. A more surprising result asserts that with three or more processes, the test&set object does not accelerate much the solution of approximate agreement. Indeed, using the round reduction theorem for augmented models (Theorem 2), we show that it still takes at least $\lceil \log_2 1/\epsilon \rceil$ rounds to solve ϵ-approximate agreement.

We start by stating an analogue of Claim 3, for the model of iterated immediate snapshot augmented with test&set. The proof of this claim appears in the full version of our paper.

Claim 4. *If $\Pi = (\mathcal{I}, \mathcal{O}, \Delta)$ is the liberal version of ϵ-approximate agreement with $n \geq 3$ processes, then the closure of Π with respect to the wait-free IIS model augmented with* test&set *is the liberal version of (2ϵ)-approximate agreement.*

Using this claim, we can derive the lower bound.

Theorem 3. *The number of rounds required for solving the ϵ-approximate agreement task in the wait-free IIS model augmented with* test&set *among $n \geq 3$ processes is at least $\lceil \log_2(1/\epsilon) \rceil$.*

Proof. We use the same reasoning as in the proof of Corollary 3, but using Theorem 2 and Claim 4. Let us fix $\epsilon > 0$ and $n \geq 3$. Consider an algorithm solving ϵ-approximate agreement in t rounds—this algorithm also solves the liberal version of ϵ-approximate agreement in t rounds. By Theorem 2 and Claim 4, there is an algorithm solving the liberal version of (2ϵ)-approximate agreement in $t - 1$ rounds, and repeating this argument for t times implies an algorithm solving the liberal version of $(2^t\epsilon)$-approximate agreement algorithm in 0 rounds. Now, note that Claim 1 still holds, since the test&set object is not used in a zero round algorithm, thus we have $2^t\epsilon \geq 1$, which immediately implies the lower bound. □

6 CONCLUSION

Our results open many interesting questions. Our asynchronous speedup theorem is inspired by an analogous speedup technique [6, 12] for the LOCAL model, and we believe that the two forms of speedup theorems shed light on the differences between asynchronous computation and computation in the LOCAL model. In particular, it seems that while in the LOCAL model it is possible to state a generic "if and only if" form of a speedup theorem, this is not the case for asynchronous computation in the wait-free model.

We have established our speedup theorem in iterated models. It would be interesting to extend it to non-iterated models, in which a register can be used multiple times. This seems especially interesting, since while the iterated and the non-iterated models are known to be equivalent with respect to task solvability [10, 11, 22, 29] (even when enriched with some more powerful objects), they are not known to be equivalent in terms of time complexity. Moreover, our speedup theorem assumes models allowing processes to run solo, as in standard wait-free models, and it would be interesting to study extensions to affine models [31] without this property, such as t-resilient models for $t < n - 1$.

We focused on some objects of consensus-number 2, such as test&set, and showed that they are useless for solving approximate agreement faster. We would be interested in extending this result to every consensus-number-2 object. However, we showed that even objects with consensus number ∞, such as binary consensus, are not useful for solving ϵ-approximate agreement (at least when $n \geq 1/\epsilon$) faster, so perhaps the answer is independent of the consensus hierarchy. And finally, it would be interesting to use the speedup theorem for problems other than consensus and approximate agreement.

REFERENCES

[1] J. Aspnes and M. Herlihy. 1990. Wait-Free Data Structures in the Asynchronous PRAM Model. In *2nd ACM Symposium on Parallel Algorithms and Architectures (SPAA)* (Island of Crete, Greece). 340–349. https://doi.org/10.1145/97444.97701

[2] Hagit Attiya, Armando Castañeda, Maurice Herlihy, and Ami Paz. 2019. Bounds on the Step and Namespace Complexity of Renaming. *SIAM J. Comput.* 48, 1 (2019), 1–32.

[3] Hagit Attiya and Faith Ellen. 2014. *Impossibility Results for Distributed Computing.* Morgan & Claypool Publishers. https://doi.org/10.2200/S00551ED1V01Y201311DCT012

[4] Hagit Attiya and Sergio Rajsbaum. 2020. Indistinguishability. *Commun. ACM* 63, 5 (2020), 90–99. https://doi.org/10.1145/3376902

[5] Hagit Attiya and Jennifer Welch. 2004. *Distributed Computing: Fundamentals, Simulations and Advanced Topics.* John Wiley & Sons, Hoboken, NJ, USA.

[6] Alkida Balliu, Sebastian Brandt, Juho Hirvonen, Dennis Olivetti, Mikaël Rabie, and Jukka Suomela. 2019. Lower Bounds for Maximal Matchings and Maximal Independent Sets. In *60th IEEE Symposium on Foundations of Computer Science (FOCS)*. 481–497. https://doi.org/10.1109/FOCS.2019.00037

[7] Paul Bastide and Pierre Fraigniaud. 2021. Brief Annoucement: On Extending Brandt's Speedup Theorem from LOCAL to Round-Based Full-Information Models. In *35th International Symposium on Distributed Computing (DISC) (LIPIcs, Vol. 209)*. Schloss Dagstuhl, 47:1–47:4. https://doi.org/10.4230/LIPIcs.DISC.2021.47

[8] Fernando Benavides and Sergio Rajsbaum. 2018. Collapsibility of read/write models using discrete morse theory. *J. Appl. Comput. Topol.* 1, 3-4 (2018), 365–396. https://doi.org/10.1007/s41468-018-0011-7

[9] Elizabeth Borowsky and Eli Gafni. 1993. Generalized FLP impossibility result for t-resilient asynchronous computations. In *25 ACM Symposium on Theory of Computing (STOC)*. 91–100. https://doi.org/10.1145/167088.167119

[10] Elizabeth Borowsky and Eli Gafni. 1997. A Simple Algorithmically Reasoned Characterization of Wait-Free Computations (Extended Abstract). In *16th ACM Symposium on Principles of Distributed Computing (PODC)*. 189–198. https://doi.org/10.1145/259380.259439

[11] Zohir Bouzid, Eli Gafni, and Petr Kuznetsov. 2014. Strong Equivalence Relations for Iterated Models. In *18th International Conference on Principles of Distributed Systems (OPODIS) (LNCS 8878)*. Springer, 139–154. https://doi.org/10.1007/978-3-319-14472-6_10

[12] Sebastian Brandt. 2019. An Automatic Speedup Theorem for Distributed Problems. In *38th ACM Symposium on Principles of Distributed Computing (PODC)*. 379–388. https://doi.org/10.1145/3293611.3331611

[13] Armando Castañeda, Pierre Fraigniaud, Ami Paz, Sergio Rajsbaum, Matthieu Roy, and Corentin Travers. 2021. A topological perspective on distributed network algorithms. *Theor. Comput. Sci.* 849 (2021), 121–137. https://doi.org/10.1016/j.tcs.2020.10.012

[14] Tushar Chandra, Vassos Hadzilacos, Prasad Jayanti, and Sam Toueg. 2005. Generalized Irreducibility of Consensus and the Equivalence of t-Resilient and Wait-Free Implementations of Consensus. *SIAM J. Comput.* 34, 2 (2005), 333–357. https://doi.org/10.1137/S0097539798344367 arXiv:https://doi.org/10.1137/S0097539798344367

[15] Danny Dolev, Nancy A. Lynch, Shlomit S. Pinter, Eugene W. Stark, and William E. Weihl. 1986. Reaching Approximate Agreement in the Presence of Faults. *J. ACM* 33, 3 (may 1986), 499?516. https://doi.org/10.1145/5925.5931

[16] Faith Ellen, Rati Gelashvili, and Leqi Zhu. 2018. Revisionist Simulations: A New Approach to Proving Space Lower Bounds. In *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing, PODC*. ACM, 61–70.

[17] Faith E. Fich and Eric Ruppert. 2003. Hundreds of impossibility results for distributed computing. *Distributed Comput.* 16, 2-3 (2003), 121–163. https://doi.org/10.1007/s00446-003-0091-y

[18] Michael J. Fischer, Nancy A. Lynch, and Mike Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *J. ACM* 32, 2 (1985), 374–382. https://doi.org/10.1145/3149.214121

[19] Pierre Fraigniaud and Ami Paz. 2020. The Topology of Local Computing in Networks. In *47th International Colloquium on Automata, Languages, and Programming (ICALP) (LIPIcs, Vol. 168)*. Schloss Dagstuhl, 128:1–128:18. https://doi.org/10.4230/LIPIcs.ICALP.2020.128

[20] Matthias Függer, Thomas Nowak, and Manfred Schwarz. 2021. Tight Bounds for Asymptotic and Approximate Consensus. *J. ACM* 68, 6, Article 46 (oct 2021), 35 pages. https://doi.org/10.1145/3485242

[21] Eli Gafni and Rachid Guerraoui. 2011. Generalized Universality. In *Proceedings of the 22nd International Conference on Concurrency Theory* (Aachen, Germany) *(CONCUR'11)*. Springer-Verlag, Berlin, Heidelberg, 17–27.

[22] Eli Gafni and Sergio Rajsbaum. 2010. Distributed Programming with Tasks. In *14th International Conference on Principles of Distributed Systems (OPODIS) (LNCS 6490)*. Springer, 205–218. https://doi.org/10.1007/978-3-642-17653-1_17

[23] Maurice Herlihy. 1991. Wait-Free Synchronization. *ACM Trans. Program. Lang. Syst.* 13, 1 (1991), 124–149.

[24] Maurice Herlihy, Dmitry N. Kozlov, and Sergio Rajsbaum. 2013. *Distributed Computing Through Combinatorial Topology.* Morgan Kaufmann.

[25] Maurice Herlihy and Sergio Rajsbaum. 1995. Algebraic Topology and Distributed Computing: A Primer. In *Computer Science Today: Recent Trends and Developments*. LNCS, Vol. 1000. Springer, 203–217. https://doi.org/10.1007/BFb0015245

[26] Maurice Herlihy, Sergio Rajsbaum, Michel Raynal, and Julien Stainer. 2017. From wait-free to arbitrary concurrent solo executions in colorless distributed computing. *Theoretical Computer Science* 683 (2017), 1–21. https://doi.org/10.1016/j.tcs.2017.04.007

[27] Maurice Herlihy and Nir Shavit. 1999. The topological structure of asynchronous computability. *J. ACM* 46, 6 (1999), 858–923. https://doi.org/10.1145/331524.331529

[28] Gunnar Hoest and Nir Shavit. 2006. Toward a Topological Characterization of Asynchronous Complexity. *SIAM J. Comput.* 36, 2 (2006), 457–497. https://doi.org/10.1137/S0097539701397412

[29] Damien Imbs, Sergio Rajsbaum, and Adrián Valle. 2015. Untangling Partial Agreement: Iterated x-consensus Simulations. In *17th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS) (LNCS 9212)*. Springer, 139–155. https://doi.org/10.1007/978-3-319-21741-3_10

[30] Dmitry Kozlov. 2015. Topology of the view complex. *Homology Homotopy Appl.* 17, 1 (2015), 307–319.

[31] Petr Kuznetsov, Thibault Rieutord, and Yuan He. 2018. An Asynchronous Computability Theorem for Fair Adversaries. In *Proceedings of the 2018 ACM Symposium on Principles of Distributed Computing* (Egham, United Kingdom) *(PODC '18)*. Association for Computing Machinery, New York, NY, USA, 387?396. https://doi.org/10.1145/3212734.3212765

[32] Nancy A. Lynch. 1989. A Hundred Impossibility Proofs for Distributed Computing. In *8th ACM Symposium on Principles of Distributed Computing (PODC)*. 1–28. https://doi.org/10.1145/72981.72982

[33] Yoram Moses and Sergio Rajsbaum. 2002. A Layered Analysis of Consensus. *SIAM J. Comput.* 31, 4 (apr 2002), 989?1021. https://doi.org/10.1137/S0097539799364006

[34] Achour Mostefaoui, Michel Raynal, and Frédéric Tronel. 2000. From Binary Consensus to Multivalued Consensus in Asynchronous Message-Passing Systems. *Inf. Process. Lett.* 73, 5-6 (2000), 207–212. https://doi.org/10.1016/S0020-0190(00)00027-2

[35] Sergio Rajsbaum. 2010. Iterated Shared Memory Models. In *LATIN 2010: Theoretical Informatics*, Alejandro López-Ortiz (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 407–416.

[36] Michel Raynal. 2018. *Fault-Tolerant Message-Passing Distributed Systems - An Algorithmic Approach.* Springer. https://doi.org/10.1007/978-3-319-94141-7

[37] Michael E. Saks and Fotios Zaharoglou. 1993. Wait-free k-set agreement is impossible: the topology of public knowledge. In *25th ACM Symposium on Theory of Computing (STOC)*. 101–110. https://doi.org/10.1145/167088.167122